Project Report

Project Title: Mutex-Yaan: A Concurrent Satellite Network and Orbital Debris Monitoring System

1. Problem Statement

The objective of this project is to design and implement a highly concurrent, real-time Central Server architecture that maintains the mathematical state of a satellite constellation. As satellites orbit, multiple remote Ground Control Stations (client programs) must connect simultaneously to pull telemetry, update positions, and schedule data transmissions.

The core challenge lies in managing highly volatile shared memory states. The system must prevent data corruption from concurrent client requests, safely log gigabytes of telemetry to disk without interleaved writes, and execute computationally heavy orbital mechanics tracking without lagging the main server threads. This necessitates the strict application of Operating System primitives, including multi-threading, mutual exclusion, reader-writer locks, POSIX file locking, TCP/IP socket programming, and multi-process IPC communication.

2. Implementation of Mandatory OS Concepts

The system strictly adheres to the mandatory OS concepts required for the project.

2.1 Role-Based Authorization

- **Concept:** Controlling access to critical infrastructure based on user privileges to ensure operational security.
- **Implementation:** The server maintains an internal mapping of users to three strict hierarchical roles: Guest (0), Payload Specialist (1), and Mission Commander (2). When a TCP connection is established, the client must authenticate. The command dispatcher validates every incoming payload against a min_role_for_cmd array. For example, a Guest requesting a CMD_ALTER_ORBIT is immediately rejected with a 403 Forbidden response, strictly restricting operations depending on permissions.

2.2 File Locking

- **Concept:** Ensuring safe file access when multiple threads or processes attempt to interact with shared log files simultaneously.
- **Implementation:** When satellites complete an orbit, they dump massive amounts of telemetry to a physical telemetry.csv file. To prevent the operating system from interleaving text lines and corrupting the audit log, the system utilizes POSIX fcntl() advisory locks. Before writing, a thread acquires an F_SETLKW (exclusive write lock),

ensuring one entire data dump finishes writing to the disk entirely before the next one is permitted to begin.

2.3 Concurrency Control

- **Concept:** Handling multiple processes/threads executing simultaneously while managing constrained resources.
- **Implementation:** Ground stations can only handle data from one satellite at a time. To prevent garbled transmission windows when two satellites fly over simultaneously, the Ground Station objects in the server are treated as Critical Sections. Synchronization is enforced using Mutexes (pthread_mutex_t). If Satellite A locks Ground Station Alpha, Satellite B's thread is put to sleep until Satellite A calls unlock(), guaranteeing mutual exclusion.

2.4 Data Consistency

- **Concept:** Maintaining the correctness of shared data structures under heavy concurrent access to prevent dirty reads and race conditions.
- **Implementation:** The central server holds a master array of all satellite XYZ coordinates and velocities. Since location-calculation threads constantly update this array while Ground Station clients constantly read it to render live maps, the system applies Reader-Writer locks (pthread_rwlock_t). This synchronization technique allows infinite concurrent readers but blocks all operations during a state update, ensuring clients only ever read fully completed, consistent data states.

2.5 Socket Programming

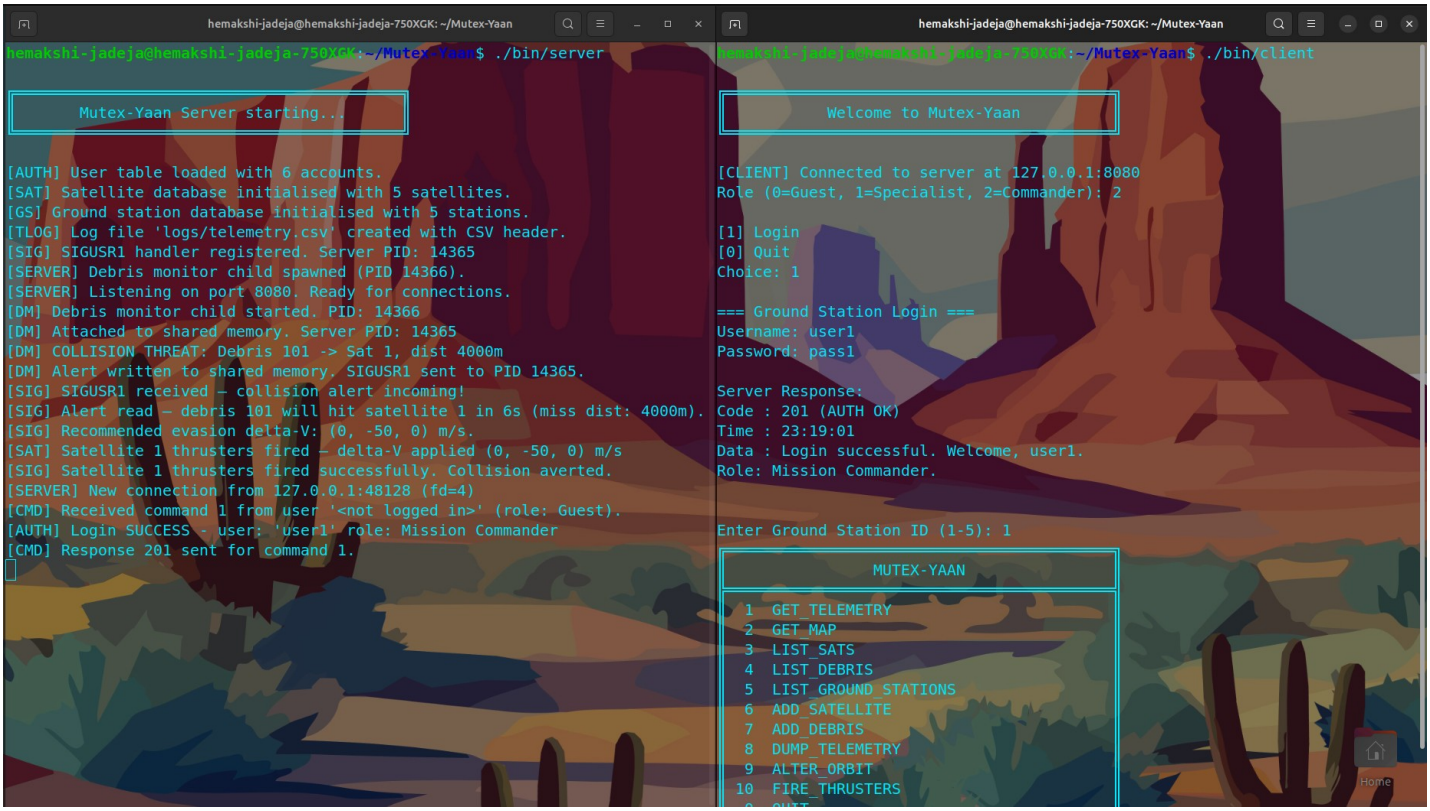
- **Concept:** Implementing a client-server communication model over network boundaries.
- **Implementation:** The core backbone of Mutex-Yaan is built on TCP/IP sockets. The server opens port 8080 and utilizes an accept() loop to spawn dedicated worker threads for each incoming Ground Station connection. Clients send structured command payloads (e.g., {"cmd": "GET_TELEMETRY", "sat_id": 4}) over the socket, which the server parses, executes, and responds to via send() and recv() system calls.

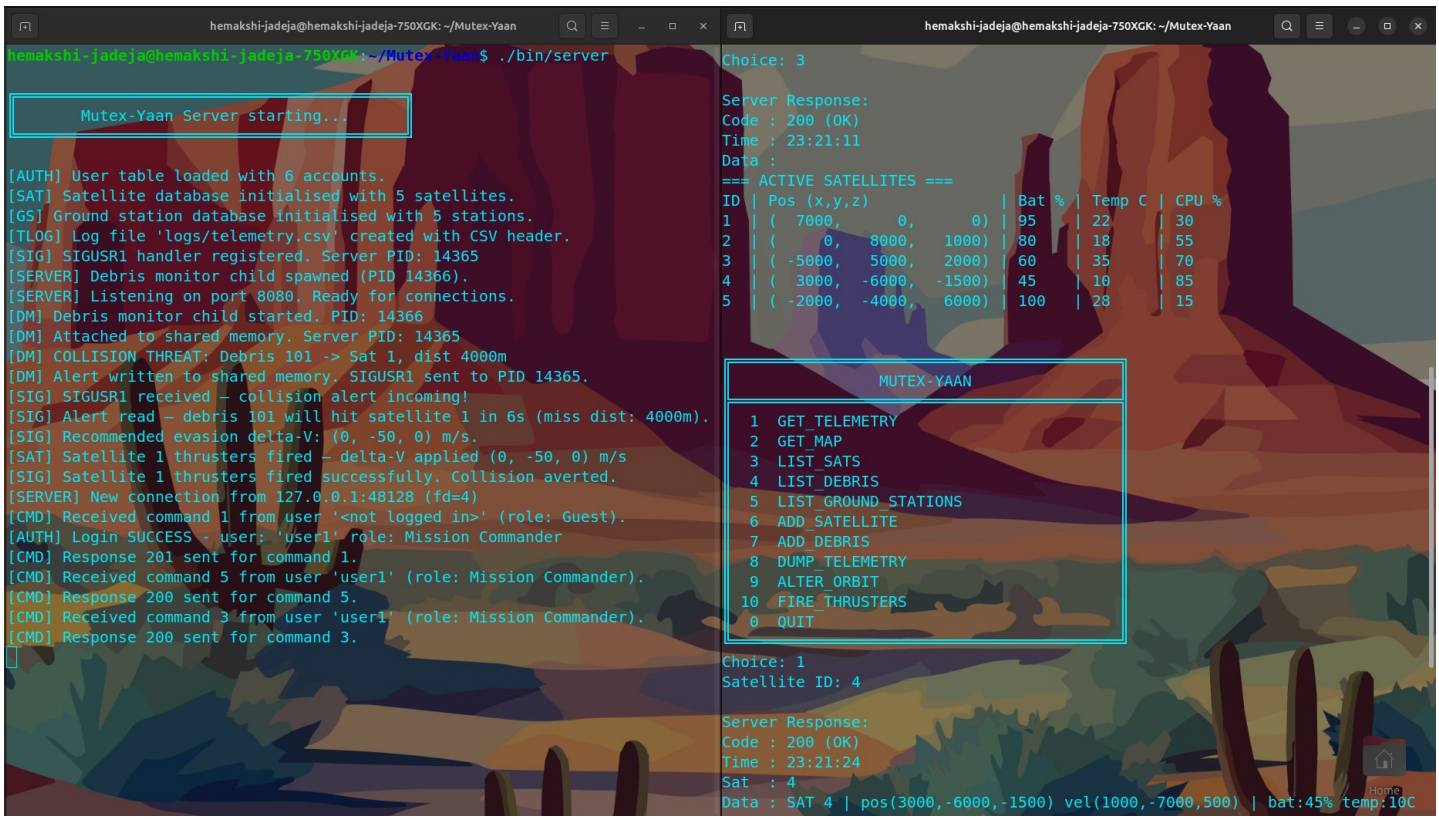
2.6 Inter-Process Communication (IPC)

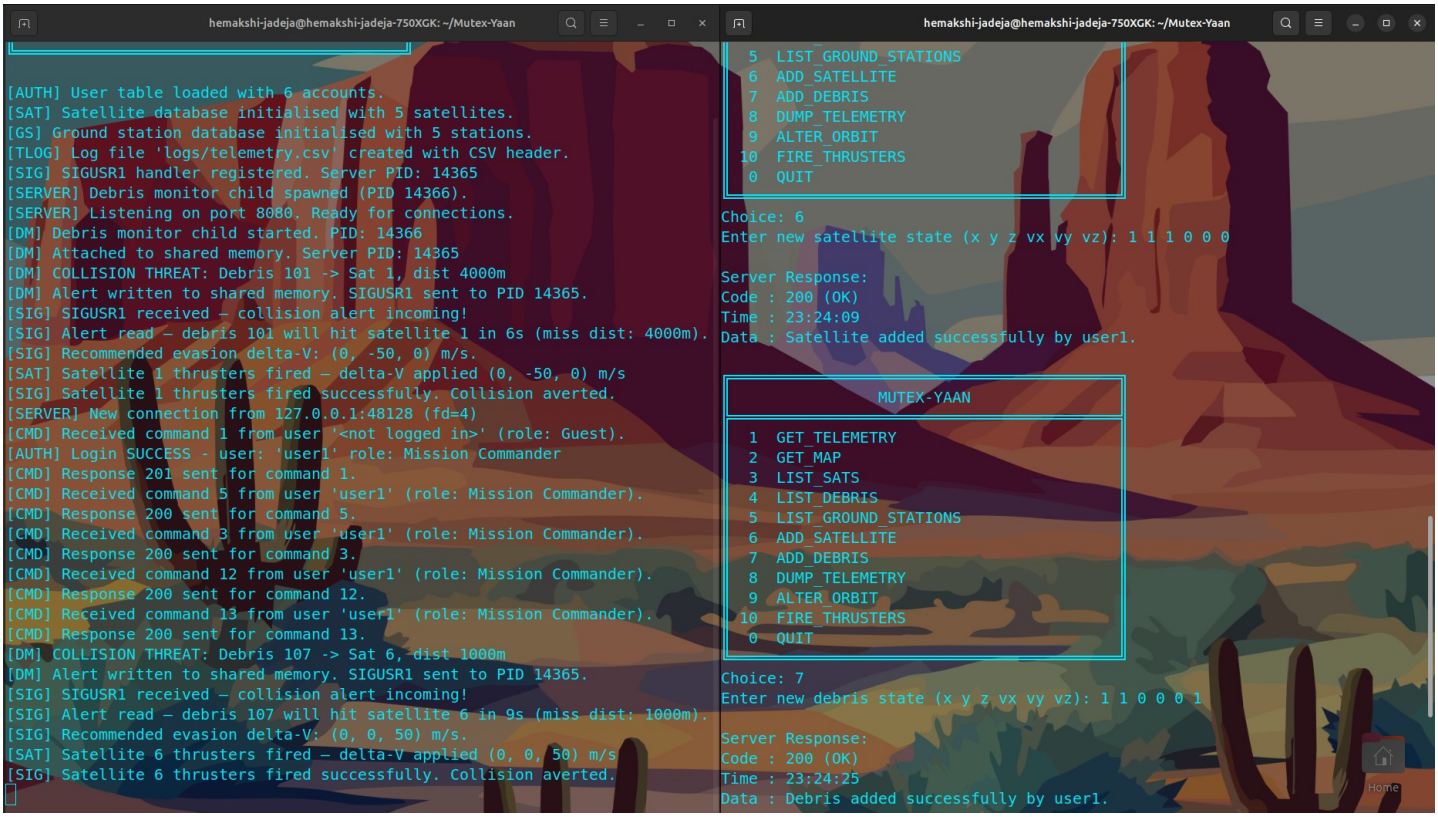
- **Concept:** Demonstrating communication between entirely separate OS processes using standard IPC mechanisms.
- **Implementation:** Calculating the trajectories of thousands of space debris fragments is highly CPU-intensive. To prevent main server lag, fork() is used to spawn a completely separate background child process dedicated strictly to orbital math. This child process communicates with the parent server using two distinct IPC mechanisms:
    1. **Shared Memory:** The child reads live satellite coordinates mapped via mmap().

2. **Signals:** If a collision threat is calculated, the child instantly fires a UNIX signal (SIGUSR1) to the parent's Process ID. The parent's signal handler catches this, interrupts its current workflow, reads the threat vectors from shared memory, and automatically fires the satellite's thrusters.

3. Execution Outputs/Screenshots







4. Challenges Faced and Solutions

This project required debugging highly complex, non-deterministic OS-level synchronization errors. The most significant technical challenges and their solutions are detailed below:

1. IPC Synchronization and Infinite Signal Loops

- **Problem:** The child process detected a collision and sent SIGUSR1. The parent server fired thrusters (changing velocity) but the child only checked coordinates, so it kept seeing the same collision and fired signals in an infinite loop.
- **Solution:** Synced velocity vectors to shared memory and implemented a mathematical convergence check (dot product of relative position and velocity). The monitor now suppresses alerts if the objects are safely diverging (moving apart).

2. Client Zombie States (Blocking I/O)

- **Problem:** Ground Station clients waiting for keyboard input (fgets) could not detect if the server abruptly crashed, leaving "zombie" terminal processes hanging indefinitely.
- **Solution:** Replaced blocking I/O with POSIX select(). This allowed the client to multiplex and monitor both STDIN (the keyboard) and the sockfd (the network) simultaneously, instantly exiting if the server dropped the TCP connection.

3. Guest Registration and Authentication Flow

- **Problem:** The command dispatcher's strict role-based permission system accidentally blocked unauthenticated users from registering, trapping them in a login failure loop.
- **Solution:** Refactored auth_check_permission() to explicitly whitelist CMD_REGISTER and CMD_LOGIN. Updated the client-side loop to offer a dedicated registration pathway specifically for the Guest role before requiring authorization.

4. Stale Shared Memory on Object Insertion

- **Problem:** When a Payload Specialist added new space debris, or a Commander manually altered a satellite's orbit, the debris_monitor child process worked with outdated coordinates until its next scan cycle, risking false alarms.
- **Solution:** Implemented a sync_positions() routine in the main server loop. Now, any state-altering command immediately forces a write-lock and flushes the fresh coordinates to the POSIX shared memory, ensuring the child process always has real-time data.

5. Edge-Case Physics in Collision Math

- **Problem:** The initial dot-product fix (dot_product < 0) successfully stopped infinite loops but accidentally caused stationary demo objects to be ignored, since their relative velocity was exactly zero.
- **Solution:** Adjusted the boundary condition to dot_product <= 0. This ensured that objects maintaining a constant, dangerous proximity inside the 5,000-meter threshold still accurately triggered the emergency IPC signal.

5. Build and Execution Instructions

To compile and run the Mutex-Yaan system, follow these steps:

Step 1: Navigate to the Project Directory

Open your terminal and navigate to the root folder of the project:

```
cd Mutex-Yaan
```

Step 2: Compile the Source Code

Use the provided Makefile to safely clean any old, compiled files and build the fresh binaries for both the server and the client:

```
make clean && make all
```

Step 3: Start the Central Server

In your current terminal, launch the server. It will initialize the shared memory, spawn the IPC background child process, and begin listening for incoming Ground Station connections on port 8080:

```
./bin/server
```

Step 4: Launch the Ground Station Client(s)

Open a **new, separate terminal window**, navigate to the same project directory, and launch the client program:

```
./bin/client
```

6. GitHub Link

https://github.com/hemakshijadeja/Mutex-Yaan